WT and DA

@Slip–1

Q.1)WriteaPHPscripttokeeptrackofnumberoftimesthewebpagehasbeenaccessed(Use Session Tracking).

Ans:

```
<?php
Session_start();
If(isset($_SESSION['count']))
{
$_SESSION['count']=$_SESSION['count']+1;
}else{
$_SESSION['count']=1;
}
echo "You have visited this page".$_SESSION['count']."Time(s)";
?>
```

Q.2)Create'Position_Salaries'Dataset.Buildalinearregressionmodelbyidentifying independentand Targetvariable.Splitthevariablesintotrainingandtestingsets.Thendividethetrainingand testingsets Intoa7:3ratio,respectivelyandprintthem.Buildasimplelinearregressionmodel.

Ans:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Create the Position_Salaries dataset
data = {
    'Position': ['CEO', 'Chairman', 'Director', 'Senior Manager', 'Junior Manager', 'Intern'],
    'Level': [1, 2, 3, 4, 5, 6],
    'Salary': [50000, 80000, 110000, 150000, 200000, 250000]
}

df = pd.DataFrame(data)

# Identify independent (X) and target (y) variables
X = df[['Level']].values   # Independent variable
y = df['Salary'].values    # Target variable
```

Edit with WPS Office

```
# Split dataset into training and testing sets (70:30 ratio)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)

# Print training and testing sets
print("X_train:\n", X_train)
print("y_train:\n", y_train)
print("X_test:\n", X_test)
print("y_test:\n", y_test)

# Build Simple Linear Regression model
regressor = LinearRegression()
regressor.fit(X_train, y_train)

# Print coefficient and intercept
print("Coefficient:", regressor.coef_)
print("Intercept:", regressor.intercept_)
```

@Slip-2

Q.1WriteaPHPscripttochangethepreferencesofyourwebpagelikefontstyle,fontsize,font color, Backgroundcolorusingcookie.Displayselectedsettingonnextwebpageandactual implementation (withnewsettings)onthirdpage(UseCookies).

Ans:

Fristpage.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Change Preferences</title>
</head>
<body>

<h1>Change Preferences</h1>

<form action="Secondpage.php" method="post">

   <label for="fontstyle">Font Style:</label>
   <select name="fontstyle" id="fontstyle">
      <option value="Arial">Arial</option>
      <option value="Times New Roman">Times New Roman</option>
      <option value="Verdana">Verdana</option>
   </select>
   <br><br>
```

Edit with WPS Office

```
    <label for="fontsize">Font Size:</label>
    <select name="fontsize" id="fontsize">
        <option value="12">12</option>
        <option value="14">14</option>
        <option value="16">16</option>
    </select>
    <br><br>

    <label for="fontcolor">Font Color:</label>
    <input type="color" name="fontcolor" id="fontcolor">
    <br><br>

    <label for="bgcolor">Background Color:</label>
    <input type="color" name="bgcolor" id="bgcolor">
    <br><br>

    <input type="submit" name="submit" value="Save">

</form>

</body>
</html>
```

Secondpage.php

```
<?php
if (isset($_POST['submit'])) {

    $fontstyle = $_POST['fontstyle'];
    $fontsize  = $_POST['fontsize'];
    $fontcolor = $_POST['fontcolor'];
    $bgcolor   = $_POST['bgcolor'];

    // Set cookies (valid for 1 day)
    setcookie('fontstyle', $fontstyle, time() + 86400);
    setcookie('fontsize', $fontsize, time() + 86400);
    setcookie('fontcolor', $fontcolor, time() + 86400);
    setcookie('bgcolor', $bgcolor, time() + 86400);

    // Redirect to third page
    header("Location: Thirdpage.php");
    exit();
}
?>
```

Edit with WPS Office

Thirdpage.php

```
<?php
// Retrieve cookie values or set defaults
$fontstyle = isset($_COOKIE['fontstyle']) ? $_COOKIE['fontstyle'] : 'Arial';
$fontsize  = isset($_COOKIE['fontsize']) ? $_COOKIE['fontsize'] : '12';
$fontcolor = isset($_COOKIE['fontcolor']) ? $_COOKIE['fontcolor'] : '#000000';
$bgcolor   = isset($_COOKIE['bgcolor']) ? $_COOKIE['bgcolor'] : '#FFFFFF';
?>

<!DOCTYPE html>
<html>
<head>
   <title>Page with New Settings</title>

   <style>
      body {
         font-family: <?php echo $fontstyle; ?>;
         font-size: <?php echo $fontsize; ?>px;
         color: <?php echo $fontcolor; ?>;
         background-color: <?php echo $bgcolor; ?>;
      }
   </style>
</head>

<body>

<h1>Page with New Settings</h1>

<p>
   Font Style: <?php echo $fontstyle; ?><br>
   Font Size: <?php echo $fontsize; ?>px<br>
   Font Color: <?php echo $fontcolor; ?><br>
   Background Color: <?php echo $bgcolor; ?>
</p>

</body>
</html>
```

Q.2)Create‘Salary’Dataset.Buildalinearregressionmodelbyidentifyingindependentand target Ariable.Splitthevariablesintotrainingandtestingsetsandprintthem.Buildasimplelinear regression Delforpredictingpurchases.

Ans:

```
import numpy as np
import pandas as pd
```

Edit with WPS Office

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Create the Salary dataset
data = {
    'YearsExperience': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Salary': [50000, 60000, 70000, 80000, 90000, 100000, 110000, 120000, 130000, 140000]
}
df = pd.DataFrame(data)

# Identify the independent (X) and target (y) variables
# X needs to be a 2D array/DataFrame, y can be a 1D array/Series
X = df[['YearsExperience']]
y = df['Salary']

# Split the variables into training and testing sets (70% train, 30% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)

# Print the training and testing sets
print("X_train:\n", X_train)
print("y_train:\n", y_train)
print("---")
print("X_test:\n", X_test)
print("y_test:\n", y_test)

# Build and train the simple linear regression model
regressor = LinearRegression()
regressor.fit(X_train, y_train)

# Print the coefficients and intercept
print("\nModel Results:")
print("Coefficients (Slope):", regressor.coef_)
print("Intercept:", regressor.intercept_)
```

@Slip-3

Q.1)WriteaPHPscripttoacceptusernameandpassword.Ifinthefirstthreechances, usernameand Passwordenterediscorrectthendisplaysecondformwith“Welcomemessage”otherwise displayerror Message.[UseSession].

Ans:

Firstpage.php

```
<?php
session_start();
```

Edit with WPS Office

```
// Initialize attempts if not set
if (!isset($_SESSION['attempts'])) {
   $_SESSION['attempts'] = 3;
}

// Check if form is submitted
if (isset($_POST['submit'])) {

   $username = $_POST['username'];
   $password = $_POST['password'];

   // Correct credentials
   $correct_username = "myusername";
   $correct_password = "mypassword";

   // Check credentials
   if ($username == $correct_username && $password == $correct_password) {

      $_SESSION['loggedin'] = true;

      // Redirect to welcome page
      header("Location: secondpage.php");
      exit();

   } else {

      $_SESSION['attempts']--;

      if ($_SESSION['attempts'] <= 0) {
         echo "<p style='color:red;'>Maximum login attempts exceeded. Please try again later.</p>";
      } else {
         echo "<p style='color:red;'>Invalid username or password. You have " . $_SESSION['attempts'] . "
attempt(s) left.</p>";
      }
   }
}
?>

<!DOCTYPE html>
<html>
<head>
   <title>Login Form</title>
</head>
<body>

<h2>Login</h2>

<form method="post">
```

Edit with WPS Office

```
    <label>Username:</label>
    <input type="text" name="username" required><br><br>

    <label>Password:</label>
    <input type="password" name="password" required><br><br>

    <input type="submit" name="submit" value="Login">
</form>

</body>
</html>
```

Secondpage.com

```
<?php
session_start();

// Check if user is logged in
if (!isset($_SESSION['loggedin'])) {
    header("Location: firstpage.php");
    exit();
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Welcome Page</title>
</head>
<body>

<h1>Welcome!</h1>

<p>You have successfully logged in.</p>

</body>
</html>
```

Q.2)Create‘User’Datasethaving5columnsnamely:UserID,Gender,Age,EstimatedSalaryand urchased.Buildalogisticregressionmodelthatcanpredictwhetheronthegivenparametera personwillbuyacarornot.

Ans:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
```

Edit with WPS Office

```
from sklearn.linear_model import LogisticRegression

# Create the User dataset
data = {
    'UserID': [1,2,3,4,5,6,7,8,9,10],
    'Gender': ['Male','Male','Female','Female','Male','Male','Female','Female','Male','Female'],
    'Age': [19,35,26,27,19,27,32,25,33,45],
    'EstimatedSalary': [19000,20000,43000,57000,76000,58000,82000,32000,69000,65000],
    'Purchased': [0,0,0,1,1,0,1,0,1,1]
}

df = pd.DataFrame(data)

# Convert categorical data (Gender) into numeric
df['Gender'] = df['Gender'].map({'Male': 0, 'Female': 1})

# Identify independent and target variables
X = df[['Gender', 'Age', 'EstimatedSalary']].values
y = df['Purchased'].values

# Split dataset into training and testing sets (70:30)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)

# Build Logistic Regression model
lr = LogisticRegression()
lr.fit(X_train, y_train)

# Predict a single observation
observation = [[0, 30, 87000]]
prediction = lr.predict(observation)
print("Single Prediction:", prediction)

# Predict multiple observations
observations = [[0,30,87000], [1,50,45000], [1,22,30000]]
predictions = lr.predict(observations)
print("Multiple Predictions:", predictions)
```

@Slip-4

Q.1)WriteaPHPscripttoacceptEmployeedetails(Eno,Ename,Address)onfirstpage.On second Pageacceptearning(Basic,DA,HRA).OnthirdpageprintEmployeeinformation(Eno,Ename, Address, Basic,DA,HRA,Total)[UseSession].

Ans:

Edit with WPS Office

Firstpage.php

```
<?php
session_start();
?>

<!DOCTYPE html>
<html>
<head>
   <title>Employee Details</title>
</head>
<body>

<h1>Employee Details</h1>

<form method="POST" action="Secondpage.php">

   <label>Employee No:</label>
   <input type="text" name="eno" required><br><br>

   <label>Employee Name:</label>
   <input type="text" name="ename" required><br><br>

   <label>Address:</label>
   <textarea name="address" required></textarea><br><br>

   <input type="submit" name="submit" value="Next">

</form>

</body>
</html>
```

Secondpage.php

```
<?php
session_start();

// Store employee details from first page
if (isset($_POST['submit'])) {
   $_SESSION['eno'] = $_POST['eno'];
   $_SESSION['ename'] = $_POST['ename'];
   $_SESSION['address'] = $_POST['address'];
}
?>

<!DOCTYPE html>
```

Edit with WPS Office

```html
<html>
<head>
   <title>Earnings</title>
</head>
<body>

<h1>Enter Earnings</h1>

<form method="POST" action="Thirdpage.php">

   <label>Basic:</label>
   <input type="number" name="basic" required><br><br>

   <label>DA:</label>
   <input type="number" name="da" required><br><br>

   <label>HRA:</label>
   <input type="number" name="hra" required><br><br>

   <input type="submit" name="submit" value="Next">

</form>

</body>
</html>
```

Thirdpage.php

```php
<?php
session_start();

// Store earnings
if (isset($_POST['submit'])) {
   $_SESSION['basic'] = $_POST['basic'];
   $_SESSION['da']    = $_POST['da'];
   $_SESSION['hra']   = $_POST['hra'];
}

// Calculate total
$total = $_SESSION['basic'] + $_SESSION['da'] + $_SESSION['hra'];
?>

<!DOCTYPE html>
<html>
<head>
   <title>Employee Information</title>
</head>
```

Edit with WPS Office

```
<body>

<h1>Employee Information</h1>

<p><strong>Employee No:</strong> <?php echo $_SESSION['eno']; ?></p>
<p><strong>Employee Name:</strong> <?php echo $_SESSION['ename']; ?></p>
<p><strong>Address:</strong> <?php echo $_SESSION['address']; ?></p>

<p><strong>Basic:</strong> <?php echo $_SESSION['basic']; ?></p>
<p><strong>DA:</strong> <?php echo $_SESSION['da']; ?></p>
<p><strong>HRA:</strong> <?php echo $_SESSION['hra']; ?></p>

<p><strong>Total Earnings:</strong> <?php echo $total; ?></p>

</body>
</html>
```

Q.2)BuildasimplelinearregressionmodelforFishSpeciesWeightPrediction.

Ans:

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

# Create dataset (Length vs Weight)
data = {
    'Length': [10, 12, 15, 18, 20, 22, 25, 28, 30, 35],
    'Weight': [100, 120, 150, 180, 200, 220, 250, 280, 300, 350]
}

df = pd.DataFrame(data)

# Independent and target variables
X = df[['Length']].values   # Independent variable
y = df['Weight'].values     # Target variable

# Build Simple Linear Regression model
model = LinearRegression()
model.fit(X, y)

# Predict weight for a new fish
new_length = [[27]]
predicted_weight = model.predict(new_length)

print("Predicted Weight:", predicted_weight)
```

Edit with WPS Office

@Slip-5

Q.1)CreateXMLfilenamed“Item.xml”withitem-name,item-rate,itemquantityStorethedetails of5 ItemsofdifferentTypes.

Ans:
Item.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<items>

  <item type="Electronics">
    <name>Television</name>
    <rate>500</rate>
    <quantity>10</quantity>
  </item>

  <item type="Clothing">
    <name>Shirt</name>
    <rate>50</rate>
    <quantity>20</quantity>
  </item>

  <item type="Grocery">
    <name>Rice</name>
    <rate>40</rate>
    <quantity>30</quantity>
  </item>

  <item type="Books">
    <name>Harry Potter and the Philosopher's Stone</name>
    <rate>20</rate>
    <quantity>50</quantity>
  </item>

  <item type="Sports">
    <name>Football</name>
    <rate>100</rate>
    <quantity>5</quantity>
  </item>

</items>
```

Q.2)Usetheirisdataset.WriteaPythonprogramtoviewsomebasicstatisticaldetailslike percentile, Mean,stdetc.Ofthespeciesof‘Iris-setosa’,‘Iris-versicolor’and‘Iris-virginica’.Applylogistic regression Onthedatasettoidentifydifferentspecies(setosa,versicolor,verginica)ofIrisflowersgiven just4 Features:sepalandpetallengthsandwidths..Findtheaccuracyofthemodel.

Edit with WPS Office

Ans:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load the Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target

# View statistical details of each species
print("Statistical details of Iris-setosa:")
print(df[df['target'] == 0].describe())

print("\nStatistical details of Iris-versicolor:")
print(df[df['target'] == 1].describe())

print("\nStatistical details of Iris-virginica:")
print(df[df['target'] == 2].describe())

# Split data into features and target
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

# Train-test split (80:20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Build Logistic Regression model
logreg = LogisticRegression(max_iter=200)
logreg.fit(X_train, y_train)

# Predictions
y_pred = logreg.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy of the Logistic Regression model:", accuracy)
```

@Slip-6

Q.1)WritePHPscripttoread“book.xml”fileintosimpleXMLobject.Displayattributesand elements.

Edit with WPS Office

(simple_xml_load_file()function).

Ans:

```
<?php

// Load XML file into SimpleXML object
$xml = simplexml_load_file("book.xml");

// Display attributes
echo "<h3>Book Attributes:</h3>";
echo "ISBN: " . $xml['isbn'] . "<br>";
echo "Publisher: " . $xml['publisher'] . "<br><br>";

// Display elements
echo "<h3>Book Elements:</h3>";
echo "Title: " . $xml->title . "<br>";
echo "Author: " . $xml->author . "<br>";
echo "Description: " . $xml->description . "<br>";
?>
```

Book.xmlfile

```
<?xml version="1.0" encoding="UTF-8"?>

<book isbn="978-3-16-148410-0" publisher="ExamplePublisher">
   <title>Example Book</title>
   <author>John Doe</author>
   <description>This is an example book.</description>
</book>
```

Q.2)Createthefollowingdatasetinpython&Convertthecategoricalvaluesintonumeric format.Apply Theapriorialgorithmontheabovedatasettogeneratethefrequentitemsetsandassociation rules.Repeat Teprocesswithdifferentmin_supvalues.
TID={1:[“bread”,”milk”],2=[“bread”,”diaper”,”beer”,”eggs”],3=[“milk”,”diaper”,”beer”,”coke”],4=[“bread”,”milk”,”diaper”,”beer”],5=[“bread”,”milk”,”diaper”,”coke”]}

Ans:

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# Create the dataset
TID = {
   1: ["bread", "milk"],
   2: ["bread", "diaper", "beer", "eggs"],
   3: ["milk", "diaper", "beer", "coke"],
```

Edit with WPS Office

```
    4: ["bread", "milk", "diaper", "beer"],
    5: ["bread", "milk", "diaper", "coke"]
}

# Convert dictionary to list of transactions
transactions = []
for key, value in TID.items():
    transactions.append(value)

# Convert categorical data into numeric format
te = TransactionEncoder()
te_ary = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_ary, columns=te.columns_)

# Apply Apriori with different min_support values
min_sup_values = [0.2, 0.4, 0.6]

for min_sup in min_sup_values:
    frequent_itemsets = apriori(df, min_support=min_sup, use_colnames=True)

    rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)

    print("\nMin_support:", min_sup)
    print("Frequent Itemsets:")
    print(frequent_itemsets)

    print("Association Rules:")
    print(rules)
```

@Slip-7

Q.1)WriteaPHPscripttoread“Movie.xml”fileandprintallMovieTitleandActorNameoffile using OMDocumentParser.“Movie.xml”fileshouldcontainfollowinginformationwithatleast5 records Wthvalues.MvieInfoMovieNo,MovieTitle,ActorName,ReReleaseYear.

Ans:

<u>Phpfile</u>

```
<?php

// Create DOMDocument object
$xml = new DOMDocument();

// Load XML file
$xml->load("Movie.xml");
```

Edit with WPS Office

```
// Get all MovieInfo nodes
$movies = $xml->getElementsByTagName("MovieInfo");

// Loop through each movie
foreach ($movies as $movie) {

   $title = $movie->getElementsByTagName("MovieTitle")->item(0)->nodeValue;
   $actor = $movie->getElementsByTagName("ActorName")->item(0)->nodeValue;

   echo "Movie Title: " . $title . "<br>";
   echo "Actor Name: " . $actor . "<br><br>";
}

?>
```

XMLfile

```
<?xml version="1.0" encoding="UTF-8"?>

<MovieList>

   <MovieInfo>
      <MovieNo>1</MovieNo>
      <MovieTitle>The Shawshank Redemption</MovieTitle>
      <ActorName>Tim Robbins</ActorName>
      <ReleaseYear>1994</ReleaseYear>
   </MovieInfo>

   <MovieInfo>
      <MovieNo>2</MovieNo>
      <MovieTitle>The Godfather</MovieTitle>
      <ActorName>Marlon Brando</ActorName>
      <ReleaseYear>1972</ReleaseYear>
   </MovieInfo>

   <MovieInfo>
      <MovieNo>3</MovieNo>
      <MovieTitle>The Dark Knight</MovieTitle>
      <ActorName>Christian Bale</ActorName>
      <ReleaseYear>2008</ReleaseYear>
   </MovieInfo>

   <MovieInfo>
      <MovieNo>4</MovieNo>
      <MovieTitle>The Godfather: Part II</MovieTitle>
      <ActorName>Al Pacino</ActorName>
      <ReleaseYear>1974</ReleaseYear>
   </MovieInfo>
```

Edit with WPS Office

```
  <MovieInfo>
    <MovieNo>5</MovieNo>
    <MovieTitle>12 Angry Men</MovieTitle>
    <ActorName>Henry Fonda</ActorName>
    <ReleaseYear>1957</ReleaseYear>
  </MovieInfo>

</MovieList>
```

Q.2)DownloadtheMarketbasketdataset.Writeapythonprogramtoreadthedatasetand displayits Information.Preprocessthedata(dropnullvaluesetc.)Convertthecategoricalvaluesinto numeric Format.Applytheapriorialgorithmontheabovedatasettogeneratethefrequentitemsetsand association Rules..

Ans:

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# Read the dataset
df = pd.read_csv('Market_Basket_Optimisation.csv', header=None)

# Display basic information
print("Dataset Info:\n")
print(df.info())

print("\nFirst 5 Rows:\n")
print(df.head())

# Preprocessing: Convert dataset into list of transactions
transactions = []
for i in range(df.shape[0]):
    transaction = []
    for j in range(df.shape[1]):
        item = df.iloc[i, j]
        if pd.notna(item):
            transaction.append(item)
    transactions.append(transaction)

# Convert categorical values into numeric (One-Hot Encoding)
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df_encoded = pd.DataFrame(te_array, columns=te.columns_)

# Apply Apriori Algorithm
frequent_itemsets = apriori(df_encoded, min_support=0.01, use_colnames=True)

# Generate Association Rules
```

Edit with WPS Office

```
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1)

# Display results
print("\nFrequent Itemsets:\n")
print(frequent_itemsets)

print("\nAssociation Rules:\n")
print(rules)
```

@Slip-8

Q.1)WriteaJavaScripttodisplaymessage‘Examsarenear,haveyoustartedpreparingfor?’ (usealert Box)andAcceptanytwonumbersfromuseranddisplayadditionoftwonumber.(UsePrompt and Confirmbox)

Ans:

```
<!DOCTYPE html>
<html>
<head>
   <title>JavaScript Example</title>
</head>
<body>

<script>

// Display message using alert box
alert("Exams are near, have you started preparing for?");

// Accept two numbers using prompt
let num1 = prompt("Enter first number:");
let num2 = prompt("Enter second number:");

// Confirm before addition
let confirmMsg = `Are you sure you want to add ${num1} and ${num2}?`;
let confirmResult = confirm(confirmMsg);

// If user confirms
if (confirmResult) {
   num1 = parseInt(num1);
   num2 = parseInt(num2);

   let sum = num1 + num2;

   alert(`The sum of ${num1} and ${num2} is ${sum}`);
}
```

Edit with WPS Office

```
</script>

</body>
</html>
```

Q.2)Downloadthegroceriesdataset.Writeapythonprogramtoreadthedatasetanddisplayits Information.Preprocessthedata(dropnullvaluesetc.)Convertthecategoricalvaluesinto numeric Format.Applytheapriorialgorithmontheabovedatasettogeneratethefrequentitemsetsand association Rules.

Ans:

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# Load the dataset
df = pd.read_csv('groceries.csv')

# Display dataset information
print("Dataset Information:\n")
print(df.info())

print("\nFirst 5 Rows:\n")
print(df.head())

# Preprocessing: Convert dataset into list of transactions
# (Assuming each row contains comma-separated items OR multiple columns)

transactions = []

# Case 1: If items are in a single column separated by commas
if df.shape[1] == 1:
    for row in df.iloc[:, 0]:
        transactions.append(str(row).split(','))

# Case 2: If items are in multiple columns
else:
    for i in range(df.shape[0]):
        transaction = []
        for j in range(df.shape[1]):
            item = df.iloc[i, j]
            if pd.notna(item):
                transaction.append(item)
        transactions.append(transaction)

# Convert categorical values into numeric format (One-Hot Encoding)
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
```

Edit with WPS Office

```
df_encoded = pd.DataFrame(te_array, columns=te.columns_)

# Apply Apriori Algorithm
frequent_itemsets = apriori(df_encoded, min_support=0.01, use_colnames=True)

# Generate Association Rules
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1)

# Display results
print("\nFrequent Itemsets:\n")
print(frequent_itemsets)

print("\nAssociation Rules:\n")
print(rules)
```

@Slip-9

Q.1)WriteaJavaScriptfunctiontovalidateusernameandpasswordforamembershipform.

Ans:

```
<!DOCTYPE html>
<html>
<head>
  <title>Membership Form</title>

  <script>
  function validateForm() {

    // Get values
    var username = document.forms["membershipForm"]["username"].value;
    var password = document.forms["membershipForm"]["password"].value;

    // Validate username
    if (username == "") {
      alert("Username must be filled out");
      return false;
    }

    // Validate password
    if (password == "") {
      alert("Password must be filled out");
      return false;
    }

    // If both are valid
```

Edit with WPS Office

```
        return true;
    }
    </script>
</head>

<body>

<h2>Membership Form</h2>

<form name="membershipForm" onsubmit="return validateForm()">

    Username: <input type="text" name="username"><br><br>

    Password: <input type="password" name="password"><br><br>

    <input type="submit" value="Submit">

</form>

</body>
</html>
```

Q.2)Createyourowntransactionsdatasetandapplytheaboveprocessonyourdataset.

Ans:

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# Create your own transaction dataset
items = ['item1', 'item2', 'item3', 'item4']

transactions = [
    ['item1', 'item2', 'item3'],
    ['item2', 'item3'],
    ['item1', 'item2', 'item4'],
    ['item1', 'item4'],
    ['item2', 'item3', 'item4'],
    ['item1', 'item3', 'item4'],
    ['item1', 'item2'],
    ['item1', 'item3'],
    ['item3', 'item4'],
    ['item2', 'item4']
]

# Convert transactions into binary matrix
te = TransactionEncoder()
```

Edit with WPS Office

```
te_array = te.fit(transactions).transform(transactions)

# Convert into DataFrame
df = pd.DataFrame(te_array, columns=te.columns_)

# Generate frequent itemsets
frequent_itemsets = apriori(df, min_support=0.3, use_colnames=True)

# Generate association rules
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)

# Print results
print("Frequent Itemsets:\n")
print(frequent_itemsets)

print("\nAssociation Rules:\n")
print(rules)
```

@Slip-10

Q.1)CreateaHTMLfiletoinserttextbeforeandafteraParagraphusingjQuery.[Hint:Use before() Andafter()].

Ans:

```
<!DOCTYPE html>
<html>
<head>
   <title>Insert Text using jQuery</title>

   <!-- jQuery CDN -->
   <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>

<body>

<h1>Insert Text Before and After Paragraph</h1>

<p id="para">This is a paragraph.</p>

<script>
$(document).ready(function(){

   $("#para").before("<b>Text inserted before the paragraph.</b><br>");

   $("#para").after("<br><b>Text inserted after the paragraph.</b>");
```

Edit with WPS Office

```
});
</script>

</body>
</html>
```

Q2).Createthefollowingdatasetinpython&Convertthecategoricalvaluesintonumeric format.Apply Theapriorialgorithmontheabovedatasettogeneratethefrequentitemsetsandassociation rules.Repeat Theprocesswithdifferentmin_supvalues.
TID={1:[“eggs”,”milk”,”bread”],2=[“eggs”,”apple”],3=[“milk”,”bread”],4=[“apple”,”milk”],5=[“milk”,”apple”,”bread”]}

Ans:

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# Create the dataset
TID = {
    1: ["eggs", "milk", "bread"],
    2: ["eggs", "apple"],
    3: ["milk", "bread"],
    4: ["apple", "milk"],
    5: ["milk", "apple", "bread"]
}

# Convert dictionary values to list of transactions
transactions = list(TID.values())

# Convert categorical values into numeric format
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_array, columns=te.columns_)

# Apply Apriori with different min_support values
min_sup_values = [0.2, 0.4, 0.6]

for min_sup in min_sup_values:
    frequent_itemsets = apriori(df, min_support=min_sup, use_colnames=True)

    rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.6)

    print("\nMin_support:", min_sup)
    print("Frequent Itemsets:\n", frequent_itemsets)

    print("Association Rules:\n", rules)
```

Edit with WPS Office

@Slip-11

Q.1)WriteaJavascriptprogramtoacceptnameofstudent,changefontcolortored,fontsize to18if Studentnameispresentotherwiseonclickingonemptytextboxdisplayimagewhichchanges itssize (Useonblur,onload,onmousehover,onmouseclick,onmouseup)
Ans:

```
<!DOCTYPE html>
<html>
<head>
   <title>JavaScript Events Example</title>

   <style>
      #name {
         font-size: 14px;
         color: black;
      }

      #img {
         display: none;
         width: 150px;
         height: 150px;
      }
   </style>
</head>

<body onload="changeImageSize()">

<h2>Enter Student Name</h2>

<input type="text" id="name"
      onblur="changeStyle()"
      onmouseover="changeSize()"
      onmouseout="resetSize()"
      onmousedown="changeColor()"
      onmouseup="resetColor()">

<br><br>

<img id="img" src="https://via.placeholder.com/150">

<script>

// onblur → check input
function changeStyle() {
   let name = document.getElementById("name").value;

   if (name != "") {
```

Edit with WPS Office

```
        document.getElementById("name").style.fontSize = "18px";
        document.getElementById("name").style.color = "red";
    } else {
        document.getElementById("img").style.display = "block";
    }
}

// onmouseover
function changeSize() {
    document.getElementById("name").style.fontSize = "16px";
}

// onmouseout
function resetSize() {
    document.getElementById("name").style.fontSize = "14px";
}

// onmousedown
function changeColor() {
    document.getElementById("name").style.color = "blue";
}

// onmouseup
function resetColor() {
    document.getElementById("name").style.color = "red";
}

// onload
function changeImageSize() {
    document.getElementById("img").style.width = "200px";
    document.getElementById("img").style.height = "200px";
}

</script>

</body>
</html>
```

Q2).Createtheabovedatasetinpython&Convertthecategoricalvaluesintonumeric format.Apply Theapriorialgorithmontheabovedatasettogeneratethefrequentitemsetsand associationrules.Repeat Theprocesswithdifferentmin_supvalues.
TID={1:[“butter”,”bread”,”milk],2=[“butter”,”flour”,”milk”,”suger”],3=[“butter”,”eggs”,”milk”,”salt”],4=[“eggs”],5=[“butter”,”flour”,”milk”,”salt”]}

Ans:

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
```

Edit with WPS Office

```
from mlxtend.frequent_patterns import apriori, association_rules

# Creating the dataset
TID = {
    1: ["butter", "bread", "milk"],
    2: ["butter", "flour", "milk", "sugar"],
    3: ["butter", "eggs", "milk", "salt"],
    4: ["eggs"],
    5: ["butter", "flour", "milk", "salt"]
}

# Convert to list of transactions
transactions = list(TID.values())

# Convert categorical values into numeric format
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_array, columns=te.columns_)

# Apply Apriori with different min_support values
min_sup_values = [0.4, 0.3, 0.2]

for min_sup in min_sup_values:
    frequent_itemsets = apriori(df, min_support=min_sup, use_colnames=True)

    print("\nFrequent Itemsets with min_support =", min_sup)
    print(frequent_itemsets)

    # Generate association rules
    rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)

    print("Association Rules with min_support =", min_sup)
    print(rules)
```

@Slip-12

Q.1)WriteAJAXprogramtoreadcontact.datfileandprintthecontentsofthefileinatabular format Whentheuserclicksonprintbutton.Contact.datfileshouldcontainsrno,name,residence number, Mobilenumber,Address.[Enteratleast3recordincontact.datfile].

Ans:

<u>Htmlfile</u>

```
<!DOCTYPE html>
<html>
<head>
```

Edit with WPS Office

```
    <title>Contact List</title>

    <!-- jQuery CDN -->
    <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>
    <script src="script.js"></script>
</head>

<body>

<button id="printBtn">Print Contacts</button>

<br><br>

<table border="1" id="contactTable">
    <thead>
        <tr>
            <th>Sr.No.</th>
            <th>Name</th>
            <th>Residence Number</th>
            <th>Mobile Number</th>
            <th>Address</th>
        </tr>
    </thead>
    <tbody>
        <!-- Data will be inserted here -->
    </tbody>
</table>

</body>
</html>
```

Ajaxfile.js

```
$(document).ready(function(){

    $("#printBtn").click(function(){

        $.ajax({
            url: "contact.dat",
            dataType: "text",

            success: function(data){

                var lines = data.split("\n");
                var tableRows = "";

                for (var i = 0; i < lines.length; i++) {

                    var columns = lines[i].split(",");
```

Edit with WPS Office

```
            if (columns.length == 5) {

                tableRows += "<tr>";

                for (var j = 0; j < columns.length; j++) {
                    tableRows += "<td>" + columns[j] + "</td>";
                }

                tableRows += "</tr>";
            }
        }

        $("#contactTable tbody").html(tableRows);
    },

    error: function() {
        alert("Error loading file");
    }

  });

 });

});
```

contact.dat

```
1,Pranay,0201234567,9876543210,Pune
2,Rahul,0207654321,9123456780,Mumbai
3,Amit,0201112222,9988776655,Nashik
```

Q.2)Create'heights-and-weights'Dataset.Buildalinearregressionmodelbyidentifying independent Andtargetvariable.Splitthevariablesintotrainingandtestingsetsandprintthem.Builda simplelinear Regressionmodelforpredictingpurchases.

Ans:

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Create a random dataset with 10 samples
heights = np.random.normal(170, 10, 10)
weights = np.random.normal(70, 5, 10)

# Combine into a DataFrame
df = pd.DataFrame({
```

Edit with WPS Office

```
    'Height': heights,
    'Weight': weights
})

# Identify independent and target variables
X = df[['Height']]   # Independent variable
y = df['Weight']     # Target variable

# Split dataset into training and testing sets (80:20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Print training and testing sets
print("X_train:\n", X_train)
print("y_train:\n", y_train)
print("X_test:\n", X_test)
print("y_test:\n", y_test)

# Build Simple Linear Regression model
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

# Print model coefficients and intercept
print("\nCoefficient:", lr_model.coef_)
print("Intercept:", lr_model.intercept_)

# Predict weights for test data
y_pred = lr_model.predict(X_test)
print("\nPredicted Weights:\n", y_pred)
```

@Slip-13

Q.1)WriteAJAXprogramwheretheuserisrequestedtowritehisorhernameinatextbox,and the Serverkeepssendingbackresponseswhiletheuseristyping.Iftheusernameisnotentered thenthe Messagedisplayedwillbe,“Stranger,pleasetellmeyourname!”.IfthenameisRohit,Virat, Dhoni, AshwinorHarbhajan,theserverrespondswith“Hello,master!”.Ifthenameisanythingelse, the Messagewillbe“,Idon’tknowyou!”.

Ans:

<u>Htmlfile</u>

```
<!DOCTYPE html>
<html>
<head>
    <title>AJAX Name Response</title>

    <!-- jQuery CDN -->
```

Edit with WPS Office

```
    <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>
</head>

<body>

<label for="name">Enter your name:</label>
<input type="text" id="name" name="name">

<br><br>

<div id="response"></div>

<script src="ajax.js"></script>

</body>
</html>
```

Ajaxfile.js

```
$(document).ready(function(){

    $("#name").on("input", function(){

        var name = $(this).val();

        $.ajax({
            url: "server.php",
            type: "POST",
            data: { name: name },

            success: function(response){
                $("#response").html(response);
            }
        });

    });

});
```

Filename:Server.php

```
<?php

$name = $_POST['name'];

// If empty
if (empty($name)) {
    echo "Stranger, please tell me your name!";
}
```

Edit with WPS Office

```
// If name is in list
elseif ($name == "Rohit" || $name == "Virat" || $name == "Dhoni" || $name == "Ashwin" || $name ==
"Harbhajan") {
   echo "Hello, master!";
}

// Otherwise
else {
   echo $name . ", I don't know you!";
}

?>
```

Q.2)DownloadnurserydatasetfromUCI.Buildalinearregressionmodelbyidentifying independent Andtargetvariable.Splitthevariablesintotrainingandtestingsetsandprintthem.Builda simplelinear Regressionmodelforpredictingpurchases.

Ans:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Load dataset from UCI
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/nursery/nursery.data"

names = ['parents','has_nurs','form','children','housing','finance','social','health','class']

dataset = pd.read_csv(url, names=names)

# Identify independent and target variables
X = dataset.drop('class', axis=1)
y = dataset['class']

# Convert categorical variables into numeric (One-Hot Encoding)
X = pd.get_dummies(X)

# Convert target variable into numeric
y = y.astype('category').cat.codes

# Split dataset into training and testing sets (80:20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Print datasets
print("X_train:\n", X_train.head())
print("y_train:\n", y_train.head())
print("X_test:\n", X_test.head())
```

Edit with WPS Office

```
print("y_test:\n", y_test.head())

# Build Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Print coefficients and intercept
print("\nIntercept:", model.intercept_)
print("Coefficients:", model.coef_)

# Predictions
y_pred = model.predict(X_test)

# Calculate Mean Squared Error
mse = np.mean((y_test - y_pred) ** 2)
print("\nMSE:", mse)
```

@Slip-14

Q.1)CreateTEACHERtableasfollowsTEACHER(tno,tname,qualification,salary).WriteAjax Programtoselectateachersnameandprinttheselectedteachersdetails.

Ans:

index.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Teacher Details</title>

   <!-- jQuery CDN -->
   <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>

<body>

<h2>Select Teacher</h2>

<select id="teacher-list">
   <option value="">--Select Teacher--</option>
   <option value="1">John Doe</option>
   <option value="2">Jane Smith</option>
   <option value="3">Bob Johnson</option>
</select>

<button id="submit-btn">Get Details</button>
```

Edit with WPS Office

```
<br><br>

<div id="details"></div>

<script>
$(document).ready(function(){

   $("#submit-btn").click(function(){

      var tno = $("#teacher-list").val();

      if (tno == "") {
         alert("Please select a teacher.");
         return;
      }

      $.ajax({
         url: "teacherdetails.php",
         method: "POST",
         data: { tno: tno },

         success: function(response){
            $("#details").html(response);
         },

         error: function(xhr){
            console.log(xhr.responseText);
         }
      });

   });

});
</script>

</body>
</html>
```

teacherdetails.php

```
<?php

// Database connection
$servername = "localhost";
$username   = "root";
$password   = "";
$dbname     = "test";   // change as per your DB
```

Edit with WPS Office

```
$conn = mysqli_connect($servername, $username, $password, $dbname);

// Check connection
if (!$conn) {
   die("Connection failed: " . mysqli_connect_error());
}

// Get teacher number
if (isset($_POST['tno'])) {

   $tno = $_POST['tno'];

   $sql = "SELECT * FROM TEACHER WHERE tno='$tno'";
   $result = mysqli_query($conn, $sql);

   if (mysqli_num_rows($result) > 0) {

      $row = mysqli_fetch_assoc($result);

      echo "Teacher Name: " . $row['tname'] . "<br>";
      echo "Qualification: " . $row['qualification'] . "<br>";
      echo "Salary: " . $row['salary'] . "<br>";

   } else {
      echo "No data found.";
   }
}

// Close connection
mysqli_close($conn);

?>
```

<u>SQL Table database</u>

```
CREATE TABLE TEACHER (tno INT PRIMARY KEY, tname VARCHAR(50), qualification VARCHAR(50), salary INT
);

INSERT INTO TEACHER VALUES
(1, 'John Doe', 'MSc', 50000),
(2, 'Jane Smith', 'PhD', 70000),
(3, 'Bob Johnson', 'MCA', 60000);
```

Q.2)Createthefollowingdatasetinpython&Convertthecategoricalvaluesintonumeric format.Apply Theapriorialgorithmontheabovedatasettogeneratethefrequentitemsetsandassociation rules.Repeat Theprocesswithdifferentmin_sup_values.
TID={1:[“apple”,”mango”,”banana”],2=[“mango”,”banana”, “cabbage”,”carrots”],3=[“mango”,”banana”,carrots”],4=[“mango”,”carrots”]}

Edit with WPS Office

Ans:

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# Create the dataset
TID = {
    1: ["apple", "mango", "banana"],
    2: ["mango", "banana", "cabbage", "carrots"],
    3: ["mango", "banana", "carrots"],
    4: ["mango", "carrots"]
}

# Convert dictionary to list of transactions
transactions = list(TID.values())

# Convert categorical values into numeric format
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_array, columns=te.columns_)

# Apply Apriori with different min_support values
min_sup_values = [0.25, 0.5, 0.75]

for min_sup in min_sup_values:
    frequent_itemsets = apriori(df, min_support=min_sup, use_colnames=True)

    print("\nFrequent Itemsets with min_support =", min_sup)
    print(frequent_itemsets)

    # Generate association rules
    rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.6)

    print("Association Rules:")
    print(rules)
```

@Slip-15

Q.1)WriteAjaxprogramtofetchsuggestionswhenisuseristypinginatextbox.(eglikegoogle Suggestions.Hintcreatearrayofsuggestionsandmatchingstringwillbedisplayed).

Ans:

```
<!DOCTYPE html>
<html>
```

Edit with WPS Office

```
<head>
   <title>Auto Suggestions Example</title>

   <script>
   function fetchSuggestions(str) {

      if (str.length == 0) {
         document.getElementById("suggestions").innerHTML = "";
         return;
      }

      var suggestions = [
         "apple", "banana", "cherry", "dates", "elderberry",
         "fig", "grape", "honeydew", "kiwi", "lemon"
      ];

      var matches = [];

      for (var i = 0; i < suggestions.length; i++) {
         if (suggestions[i].toLowerCase().startsWith(str.toLowerCase())) {
            matches.push(suggestions[i]);
         }
      }

      if (matches.length > 0) {
         document.getElementById("suggestions").innerHTML = matches.join("<br>");
      } else {
         document.getElementById("suggestions").innerHTML = "No suggestions found";
      }
   }
   </script>

</head>

<body>

<h2>Search Fruit:</h2>

<input type="text" onkeyup="fetchSuggestions(this.value)">

<br><br>

<div id="suggestions"></div>

</body>
</html>
```

Q.2)Createthefollowingdatasetinpython&Convertthecategoricalvaluesintonumeric format.Apply

Edit with WPS Office

Theapriorialgorithmontheabovedatasettogeneratethefrequentitemsetsandassociation rules.Repeat Theprocesswithdifferentmin_supvalues.
No|Company|model|year
Tata. Nexon. 2017
MG. Astor. 2021
Kia. Seltos. 2019
Hyundai. Creta. 2015

Ans:

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# Create the dataset
data = [
    ["Tata", "Nexon", "2017"],
    ["MG", "Astor", "2021"],
    ["Kia", "Seltos", "2019"],
    ["Hyundai", "Creta", "2015"]
]

# Convert into transaction format
transactions = data

# Convert categorical values into numeric (One-Hot Encoding)
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_array, columns=te.columns_)

# Apply Apriori with different min_support values
min_sup_values = [0.25, 0.5, 0.75]

for min_sup in min_sup_values:
    frequent_itemsets = apriori(df, min_support=min_sup, use_colnames=True)

    print("\nFrequent Itemsets with min_support =", min_sup)
    print(frequent_itemsets)

    # Generate association rules
    rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)

    print("Association Rules:")
    print(rules)
```

@Slip-16

Edit with WPS Office

Q.1)WriteAjaxprogramtogetbookdetailsfromXMLfilewhenuserselectabookname. CreateXML Fileforstoringdetailsofbook(title,author,year,price).

Ans:

Xmlfilebook_details.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<books>

  <book>
    <title>The Great Gatsby</title>
    <author>F. Scott Fitzgerald</author>
    <year>1925</year>
    <price>10.99</price>
  </book>

  <book>
    <title>To Kill a Mockingbird</title>
    <author>Harper Lee</author>
    <year>1960</year>
    <price>8.99</price>
  </book>

  <book>
    <title>1984</title>
    <author>George Orwell</author>
    <year>1949</year>
    <price>6.99</price>
  </book>

  <book>
    <title>Pride and Prejudice</title>
    <author>Jane Austen</author>
    <year>1813</year>
    <price>7.99</price>
  </book>

</books>
```

Ajaxfile.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Book Details</title>

  <!-- jQuery CDN -->
```

Edit with WPS Office

```
  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>

  <script>
  $(document).ready(function(){

    $("#bookList").change(function(){

      var selectedBook = $(this).val();

      $.ajax({
        url: "book_details.xml",
        dataType: "xml",

        success: function(xml){

          var output = "";

          $(xml).find("book").each(function(){

            var title = $(this).find("title").text();

            if (title == selectedBook) {

              var author = $(this).find("author").text();
              var year   = $(this).find("year").text();
              var price  = $(this).find("price").text();

              output = "Author: " + author + "<br>" +
                       "Year: " + year + "<br>" +
                       "Price: " + price;
            }
          });

          $("#details").html(output);
        }
      });

    });

  });
  </script>

</head>

<body>

<h2>Select Book</h2>
```

Edit with WPS Office

```
<select id="bookList">
   <option value="">--Select Book--</option>
   <option>The Great Gatsby</option>
   <option>To Kill a Mockingbird</option>
   <option>1984</option>
   <option>Pride and Prejudice</option>
</select>

<br><br>

<div id="details"></div>

</body>
</html>
```

Q2).Consideranytextparagraph.Preprocessthetexttoremoveanyspecialcharactersand digits. Generatethesummaryusingextractivesummarizationpprocess.

Ans:

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import sent_tokenize, word_tokenize
from heapq import nlargest

# Sample text paragraph
text = """Natural language processing (NLP) is a subfield of linguistics, computer science,
information engineering, and artificial intelligence concerned with the interactions between
computers and human languages. It focuses on how to program computers to process and analyze
large amounts of natural language data. Challenges in NLP include speech recognition,
natural language understanding, and natural language generation. The history of NLP started
in the 1950s, although earlier work exists."""

# Remove special characters and digits
clean_text = re.sub(r'[^a-zA-Z\s]', '', text)

# Tokenize into sentences
sentences = sent_tokenize(clean_text)

# Prepare stopwords
stop_words = set(stopwords.words('english'))

# Tokenize words and remove stopwords
words = word_tokenize(clean_text.lower())
words = [word for word in words if word not in stop_words]

# Calculate word frequency
```

Edit with WPS Office

```
word_freq = {}
for word in words:
    word_freq[word] = word_freq.get(word, 0) + 1

# Score sentences
sentence_scores = {}
for sentence in sentences:
    for word in word_tokenize(sentence.lower()):
        if word in word_freq:
            if len(sentence.split()) < 30:
                sentence_scores[sentence] = sentence_scores.get(sentence, 0) + word_freq[word]

# Select top 3 sentences
summary_sentences = nlargest(3, sentence_scores, key=sentence_scores.get)

# Generate summary
summary = ' '.join(summary_sentences)

print("Summary:\n")
print(summary)
```

@Slip-17

Q.1)WriteaJavaScriptProgramtoshowHelloGoodMorningmessageonloadeventusing alertbox AnddisplaytheStudentregistrationfrom.

Ans:

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Registration Form</title>

    <script>
    window.onload = function() {
        alert("Hello Good Morning!");
    };
    </script>
</head>

<body>

<h1>Student Registration Form</h1>

<form>
```

Edit with WPS Office

```
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required>
    <br><br>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>
    <br><br>

    <label for="phone">Phone:</label>
    <input type="tel" id="phone" name="phone" required>
    <br><br>

    <label for="address">Address:</label>
    <textarea id="address" name="address" required></textarea>
    <br><br>

    <input type="submit" value="Submit">

</form>

</body>
</html>
```

Q.2)Considertextparagraph.So,keepworking.Keepstriving.Nevergiveup.Falldownseven times,get Upeight.Easeisagreaterthreattoprogressthanhardship.Easeisagreaterthreattoprogress than Hardship.So,keepmoving,keepgrowing,keeplearning.Seeyouatwork.Preprocessthetextto remove Anyspecialcharactersanddigits.Generatethesummaryusingextractivesummarization process.

Ans:

```
import re
from nltk.tokenize import sent_tokenize

# Text paragraph
text = """So, keep working. Keep striving. Never give up.
Fall down seven times, get up eight.
Ease is a greater threat to progress than hardship.
Ease is a greater threat to progress than hardship.
So, keep moving, keep growing, keep learning.
See you at work."""

# Remove special characters and digits
clean_text = re.sub(r'[^A-Za-z\s]', '', text)

# Tokenize sentences
sentences = sent_tokenize(clean_text)

# Calculate score of each sentence based on number of words
```

Edit with WPS Office

```
scores = {}
for sentence in sentences:
    words = sentence.split()
    score = len(words)
    scores[sentence] = score

# Sort sentences based on score (descending)
sorted_sentences = sorted(scores.items(), key=lambda x: x[1], reverse=True)

# Select top 2 sentences
summary_sentences = [sentence[0] for sentence in sorted_sentences[:2]]

# Generate summary
summary = " ".join(summary_sentences)

# Print summary
print("Summary:\n")
print(summary)
```

@Slip-18

Q.1)WriteaJavaScriptProgramtoprintFibonaccinumbersononclickevent.

Ans:

```
<!DOCTYPE html>
<html>
<head>
    <title>Fibonacci Numbers</title>

    <script>
    function generateFibonacci() {

        // Get input value
        var input = document.getElementById("inputNumber").value;
        var n = parseInt(input);

        var output = document.getElementById("output");

        // Create array for Fibonacci
        var fib = [];

        fib[0] = 0;
        fib[1] = 1;

        for (var i = 2; i <= n; i++) {
```

Edit with WPS Office

```
        fib[i] = fib[i - 1] + fib[i - 2];
      }

      // Display result
      output.innerHTML = "Fibonacci sequence up to " + n + ": " + fib.join(", ");
    }
    </script>

</head>

<body>

<h1>Fibonacci Numbers</h1>

<p>Enter a number:</p>

<input type="text" id="inputNumber">

<br><br>

<button onclick="generateFibonacci()">Generate Fibonacci</button>

<p id="output"></p>

</body>
</html>
```

Q.2)Consideranytextparagraph.Removethestopwords.Tokenizetheparagraphtoextract wordsand Sentences.Calculatethewordfrequencydistributionandplotthefrequencies.Plotthe wordcloudofthe Txt.

Ans:

```
# Install libraries (run once in notebook)
# !pip install nltk matplotlib wordcloud

import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.probability import FreqDist
import matplotlib.pyplot as plt
from wordcloud import WordCloud

# Download required resources
nltk.download('punkt')
nltk.download('stopwords')

# Define text paragraph
text = """Lorem ipsum dolor sit amet, consectetur adipiscing elit.
```

Edit with WPS Office

```
Sed tristique ante et velit vestibulum, vel pharetra orci iaculis.
Nullam mattis risus quis augue tincidunt rhoncus. Morbi varius,
arcu vitae scelerisque laoreet, magna est imperdiet quam,
sit amet ultrices lectus justo id enim."""

# Tokenize into words and sentences
words = word_tokenize(text.lower())
sentences = sent_tokenize(text)

# Remove stopwords
stop_words = set(stopwords.words('english'))
filtered_words = [word for word in words if word.isalpha() and word not in stop_words]

# Calculate frequency distribution
fdist = FreqDist(filtered_words)

# Plot frequency distribution
plt.figure()
fdist.plot(30, cumulative=False)
plt.show()

# Generate WordCloud
wordcloud = WordCloud(width=800, height=800,
                      background_color='white',
                      stopwords=stop_words,
                      min_font_size=10).generate(" ".join(filtered_words))

# Plot WordCloud
plt.figure(figsize=(8,8))
plt.imshow(wordcloud)
plt.axis("off")
plt.tight_layout(pad=0)
plt.show()
```

@Slip-19

Q.1)WriteaJavaScriptProgramtovalidateusernameandpasswordononSubmitevent.

Ans:

```
<!DOCTYPE html>
<html>
<head>
   <title>Validate Username and Password</title>

   <script>
```

Edit with WPS Office

```
  function validateForm() {

    var username = document.forms["myForm"]["username"].value;
    var password = document.forms["myForm"]["password"].value;

    if (username == "") {
      alert("Username must be filled out");
      return false;
    }

    if (password == "") {
      alert("Password must be filled out");
      return false;
    }

    return true; // allow form submission
  }
  </script>
</head>

<body>

<h2>Validate Username and Password</h2>

<form name="myForm" onsubmit="return validateForm()" method="post">

  <label for="username">Username:</label>
  <input type="text" id="username" name="username">
  <br><br>

  <label for="password">Password:</label>
  <input type="password" id="password" name="password">
  <br><br>

  <input type="submit" value="Submit">

</form>

</body>
</html>
```

Q.2)Downloadthemovie_review.csvdatasetfromKagglebyusingthefollowinglink :https://www.kaggle.com/nltkdata/movie-review/version/3?select=movie_review.csvto perform Sentimentanalysisonabovedatasetandcreateawordcloud.

Ans:

```
import pandas as pd
```

Edit with WPS Office

```
from textblob import TextBlob
from wordcloud import WordCloud, STOPWORDS
import matplotlib.pyplot as plt

# Load the dataset
df = pd.read_csv('movie_review.csv')

# Perform sentiment analysis using TextBlob
df['Sentiment'] = df['Review'].apply(lambda x: TextBlob(str(x)).sentiment.polarity)

# Filter positive reviews
pos_df = df[df['Sentiment'] > 0.2]

# Create WordCloud for positive reviews
text = " ".join(pos_df['Review'].astype(str))

wordcloud = WordCloud(width=800, height=800,
                      background_color='white',
                      stopwords=STOPWORDS,
                      min_font_size=10).generate(text)

# Plot WordCloud
plt.figure(figsize=(8,8))
plt.imshow(wordcloud)
plt.axis("off")
plt.tight_layout(pad=0)
plt.show()
```

@Slip-20

Q.1)createastudent.xmlfilecontainingatleast5studentinformation.

Ans:

student.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<students>

  <student>
    <name>John Doe</name>
    <age>21</age>
    <gender>Male</gender>
    <major>Computer Science</major>
    <gpa>3.8</gpa>
  </student>
```

Edit with WPS Office

```
  <student>
    <name>Jane Smith</name>
    <age>19</age>
    <gender>Female</gender>
    <major>Business</major>
    <gpa>3.5</gpa>
  </student>

  <student>
    <name>Tom Johnson</name>
    <age>20</age>
    <gender>Male</gender>
    <major>Engineering</major>
    <gpa>3.2</gpa>
  </student>

  <student>
    <name>Sara Lee</name>
    <age>22</age>
    <gender>Female</gender>
    <major>Psychology</major>
    <gpa>3.6</gpa>
  </student>

  <student>
    <name>Mike Brown</name>
    <age>18</age>
    <gender>Male</gender>
    <major>Education</major>
    <gpa>3.4</gpa>
  </student>

</students>
```

Q.2)Considertextparagraph.”””Helloall,WelcometoPythonProgrammingAcademy.Python ProgrammingAcademyisaniceplatformtolearnnewprogrammingskills.Itisdifficulttoget enrolled InthisAcademy.”””Removethestopwords.

Ans:

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Download required data
nltk.download('punkt')
nltk.download('stopwords')
```

Edit with WPS Office

```
# Text paragraph
text = """Hello all, Welcome to Python Programming Academy.
Python Programming Academy is a nice platform to learn new programming skills.
It is difficult to get enrolled in this Academy."""

# Tokenize the text
tokens = word_tokenize(text)

# Remove stopwords
stop_words = set(stopwords.words('english'))
filtered_tokens = [word for word in tokens if word.lower() not in stop_words]

# Print result
print("Filtered Tokens:\n")
print(filtered_tokens)
```

@Slip-21

Q.1)AddaJavaScriptFileinCodeigniter.TheJavascriptcodeshouldcheckwhetheranumber is Positiveornegative.

Ans:

htmlfile

```
<!DOCTYPE html>
<html>
<head>
   <title>Number Check</title>

   <!-- Load external JS file using base_url -->
   <script src="<?php echo base_url('js/numberCheck.js'); ?>"></script>
</head>

<body>

<h1>Number Check</h1>

<p>Enter a number to check:</p>

<input type="number" id="num">

<br><br>

<button onclick="checkNumber(document.getElementById('num').value)">
   Check
```

Edit with WPS Office

```
</button>

</body>
</html>
```

numberCheck.js

```
function checkNumber(num) {

   num = parseFloat(num);

   if (num > 0) {
      alert("The number is positive.");
   }
   else if (num < 0) {
      alert("The number is negative.");
   }
   else {
      alert("The number is zero.");
   }
}
```

Q.2)BuildasimplelinearregressionmodelforUserData.

Ans:

```
Importpandasaspd
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# 1. Collect data
data = pd.read_csv('user_data.csv')

# 2. Preprocess data
data.dropna(inplace=True)

# Independent and target variables
X = data[['age']]       # Independent variable
y = data['income']      # Target variable

# 3. Split data (80:20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# 4. Train the model
regressor = LinearRegression()
regressor.fit(X_train, y_train)
```

Edit with WPS Office

```
# 5. Predict values
y_pred = regressor.predict(X_test)

# 6. Evaluate model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R-squared:", r2)

# 7. Visualize results
plt.scatter(X_test, y_test)
plt.plot(X_test, y_pred)
plt.show()
```

@Slip-22

Q.1)Createatablestudenthavingattributes(rollno,name,class).Usingcodeigniter,connectto the Databaseandinsert5recodesinit.
Ans:

application/config/database.php

```
$db['default'] = array(
   'dsn'      => '',
   'hostname' => 'localhost',
   'username' => 'root',
   'password' => '',
   'database' => 'test',
   'dbdriver' => 'mysqli',
);
```

Student_model.php

```
<?php
class Student_model extends CI_Model {

   // Create table
   public function create_table() {
      $query = "CREATE TABLE IF NOT EXISTS student (
         rollno INT PRIMARY KEY,
         name VARCHAR(50),
         class VARCHAR(10)
      )";

      return $this->db->query($query);
   }
```

Edit with WPS Office

```
    // Insert records
    public function insert_students() {

        $data = array(
            array('rollno' => 1, 'name' => 'John Doe', 'class' => '10A'),
            array('rollno' => 2, 'name' => 'Jane Smith', 'class' => '9B'),
            array('rollno' => 3, 'name' => 'Bob Johnson', 'class' => '11C'),
            array('rollno' => 4, 'name' => 'Sarah Lee', 'class' => '12D'),
            array('rollno' => 5, 'name' => 'Tom Brown', 'class' => '8E')
        );

        return $this->db->insert_batch('student', $data);
    }
}
?>
```

Student.php

```
<?php
class Student extends CI_Controller {

    public function index() {

        $this->load->model('Student_model');

        // Create table
        $this->Student_model->create_table();

        // Insert records
        $this->Student_model->insert_students();

        echo "Table created and 5 records inserted successfully.";
    }
}
?>
```

Q2).Consideranytextparagraph.Removethestopwords.

Ans:

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Download required resources
nltk.download('punkt')
nltk.download('stopwords')
```

Edit with WPS Office

```
# Sample text paragraph
text = """Hello all, Welcome to Python Programming Academy.
Python Programming Academy is a nice platform to learn new programming skills.
It is difficult to get enrolled in this Academy."""

# Tokenize the text
words = word_tokenize(text)

# Define stopwords
stop_words = set(stopwords.words('english'))

# Remove stopwords
filtered_words = [word for word in words if word.lower() not in stop_words]

# Join filtered words into sentence
filtered_sentence = " ".join(filtered_words)

print("Filtered Sentence:\n")
print(filtered_sentence)
```

@Slip-23

Q.1)Createatablestudenthavingattributes(rollno,name,class)containingatleast5recodes. Using Codeigniter,displayallitsrecords.

Ans:

<u>Database Table (SQL)</u>

```
CREATE TABLE student (
   rollno INT PRIMARY KEY,
   name VARCHAR(50),
   class VARCHAR(10)
);

INSERT INTO student VALUES
(1, 'John Doe', '10A'),
(2, 'Jane Smith', '9B'),
(3, 'Bob Johnson', '11C'),
(4, 'Sarah Lee', '12D'),
(5, 'Tom Brown', '8E');
```

<u>Student_model.php</u>

```
<?php
class Student_model extends CI_Model {

   // Fetch all records
```

Edit with WPS Office

```
    public function get_students() {
        return $this->db->get('student')->result();
    }
}
?>
```

Student.php

```
<?php
class Student extends CI_Controller {

    public function index() {

        $this->load->model('Student_model');

        $data['students'] = $this->Student_model->get_students();

        $this->load->view('student_view', $data);
    }
}
?>
```

student_view.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Records</title>
</head>
<body>

<h2>Student Records</h2>

<table border="1">
    <tr>
        <th>Roll No</th>
        <th>Name</th>
        <th>Class</th>
    </tr>

    <?php foreach ($students as $row) { ?>
    <tr>
        <td><?php echo $row->rollno; ?></td>
        <td><?php echo $row->name; ?></td>
        <td><?php echo $row->class; ?></td>
    </tr>
    <?php } ?>

</table>
```

Edit with WPS Office

```
</body>
</html>
```

Q2).Consideranytextparagraph.Preprocessthetexttoremoveanyspecialcharactersand Digits.

Ans:

```
import re

# Sample text paragraph
text = "Hello,#world123! This is a sample text paragraph. It contains special characters and 5 digits."

# Remove special characters and digits
processed_text = re.sub(r'[^a-zA-Z\s]', '', text)

# Print result
print(processed_text)
```

@Slip-24

Q.1)WriteaPHPscripttocreatestudent.xmlfilewhichcontainsstudentrollno,name,address, college Andcourse.Printstudentsdetailofspecificcourseintabularformatafteracceptingcourseas input.

Ans:

```
<?php

// Step 1: Define student data
$students = array(
   array("rollno"=>1,"name"=>"John Doe","address"=>"123 Main St","college"=>"ABC College","course"=>"Computer Science"),
   array("rollno"=>2,"name"=>"Jane Smith","address"=>"456 Main St","college"=>"DEF College","course"=>"Information Technology"),
   array("rollno"=>3,"name"=>"Bob Johnson","address"=>"789 Main St","college"=>"GHI College","course"=>"Business Administration"),
   array("rollno"=>4,"name"=>"Sarah Lee","address"=>"101 Main St","college"=>"JKL College","course"=>"Marketing"),
   array("rollno"=>5,"name"=>"Tom Brown","address"=>"121 Main St","college"=>"MNO College","course"=>"Computer Science")
);

// Step 2: Create XML file
$xml = new SimpleXMLElement('<students></students>');

foreach ($students as $s) {
   $student = $xml->addChild('student');
```

Edit with WPS Office

```
    $student->addChild('rollno', $s['rollno']);
    $student->addChild('name', $s['name']);
    $student->addChild('address', $s['address']);
    $student->addChild('college', $s['college']);
    $student->addChild('course', $s['course']);
}

// Save XML
$xml->asXML("student.xml");

// Step 3: Get input course
$course = isset($_POST['course']) ? $_POST['course'] : "";

// Step 4: Load XML
$xml = simplexml_load_file("student.xml");

// Step 5: Filter using XPath
$filtered = $xml->xpath("//student[course='$course']");

?>

<!DOCTYPE html>
<html>
<head>
    <title>Student Details</title>
</head>
<body>

<h2>Search by Course</h2>

<form method="post">
    Enter Course: <input type="text" name="course">
    <input type="submit" value="Search">
</form>

<br>

<?php
// Step 6: Display table
if ($course != "") {

    echo "<table border='1'>";
    echo "<tr><th>Roll No</th><th>Name</th><th>Address</th><th>College</th><th>Course</th></tr>";

    foreach ($filtered as $student) {
        echo "<tr>";
        echo "<td>{$student->rollno}</td>";
        echo "<td>{$student->name}</td>";
```

Edit with WPS Office

```
        echo "<td>{$student->address}</td>";
        echo "<td>{$student->college}</td>";
        echo "<td>{$student->course}</td>";
        echo "</tr>";
    }

    echo "</table>";
}
?>

</body>
</html>
```

Q.2)Considerthefollowingdataset:
https://www.kaggle.com/datasets/datasnaek/youtubenew?select=INvideos.csv
WriteaPythonscriptforthefollowing: i.
Readthedatasetandperformdatacleaningoperationsonit. ii.
ii.Findthetotalviews,totallikes,totaldislikesandcommentcount.

Ans:

```
import pandas as pd

# Read dataset
df = pd.read_csv("INvideos.csv")

# Data Cleaning: Drop unnecessary columns
df = df.drop([
    'video_id', 'trending_date', 'channel_title', 'category_id',
    'publish_time', 'tags', 'thumbnail_link',
    'comments_disabled', 'ratings_disabled', 'video_error_or_removed'
], axis=1)

# Convert required columns to numeric (safe conversion)
cols = ['views', 'likes', 'dislikes', 'comment_count']
for col in cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# Remove null values after conversion
df.dropna(inplace=True)

# Convert to integer
df[cols] = df[cols].astype(int)

# Find totals
total_views = df['views'].sum()
total_likes = df['likes'].sum()
total_dislikes = df['dislikes'].sum()
```

Edit with WPS Office

```
total_comments = df['comment_count'].sum()

# Print results
print("Total Views:", total_views)
print("Total Likes:", total_likes)
print("Total Dislikes:", total_dislikes)
print("Total Comments:", total_comments)
```

@Slip-25

Q.1)Writeascripttocreate“cricket.xml”filewithmultipleelementsasshownbelow:
<CricketTeam>
<Teamcountry=”Australia”>
<player>____</player>
<runs>_____</runs>
<wicket>____</wicket>
</Team>
</CricketTeam>
Writeascripttoaddmultipleelementsin“cricket.xml”fileofcategory,country=”India”.

Ans:

```
<?php

// Create DOM Document
$doc = new DOMDocument("1.0", "UTF-8");
$doc->formatOutput = true;

// Root element
$cricketTeam = $doc->createElement("CricketTeam");

// -------- Team 1: Australia --------
$team1 = $doc->createElement("Team");
$team1->setAttribute("country", "Australia");

$team1->appendChild($doc->createElement("player", "Steve Smith"));
$team1->appendChild($doc->createElement("runs", "7090"));
$team1->appendChild($doc->createElement("wicket", "17"));

$cricketTeam->appendChild($team1);

// -------- Team 2: India --------
$team2 = $doc->createElement("Team");
$team2->setAttribute("country", "India");

$team2->appendChild($doc->createElement("player", "Virat Kohli"));
```

Edit with WPS Office

```
$team2->appendChild($doc->createElement("runs", "12169"));
$team2->appendChild($doc->createElement("wicket", "4"));

// Additional element (category)
$team2->appendChild($doc->createElement("category", "Captain"));

$cricketTeam->appendChild($team2);

// Append root to document
$doc->appendChild($cricketTeam);

// Save XML file
$doc->save("cricket.xml");

echo "cricket.xml file created successfully!";

?>
```

cricket.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<CricketTeam>
  <Team country="Australia">
    <player>Steve Smith</player>
    <runs>7090</runs>
    <wicket>17</wicket>
  </Team>
  <Team country="India">
    <player>Virat Kohli</player>
    <runs>12169</runs>
    <wicket>4</wicket>
    <category>Captain</category>
  </Team>
</CricketTeam>
```

Q.2)Considerthefollowingdataset:https://www.kaggle.com/datasets/seungguini/youtubecommentsfor-covid19-relatedvideos?select=covid_2021_1.csv WriteaPythonscriptforthefollowing: i. Readthedatasetandperformdatacleaningoperationsonit. ii. ii.Tokenizethecommentsinwords.Iii.Performsentimentanalysisandfindthepercentageof positive,negativeandneutralcomments..

Ans:

```
import pandas as pd
import nltk
from nltk.tokenize import word_tokenize
from nltk.sentiment.vader import SentimentIntensityAnalyzer

# Download required resources
```

Edit with WPS Office

```
nltk.download('punkt')
nltk.download('vader_lexicon')

# Read dataset
df = pd.read_csv("covid_2021_1.csv")

# Data Cleaning
df.dropna(inplace=True)
df.drop_duplicates(subset='Comment', inplace=True)

# Tokenize comments into words
df['tokens'] = df['Comment'].apply(word_tokenize)

# Sentiment Analysis using VADER
sia = SentimentIntensityAnalyzer()
df['sentiment'] = df['Comment'].apply(lambda x: sia.polarity_scores(str(x))['compound'])

# Calculate counts
total_comments = len(df)

positive_comments = len(df[df['sentiment'] > 0])
negative_comments = len(df[df['sentiment'] < 0])
neutral_comments  = len(df[df['sentiment'] == 0])

# Calculate percentages
positive_percentage = (positive_comments / total_comments) * 100
negative_percentage = (negative_comments / total_comments) * 100
neutral_percentage  = (neutral_comments / total_comments) * 100

# Print results
print("Total Comments:", total_comments)
print("Positive Comments:", positive_comments, f"({positive_percentage:.2f}%)")
print("Negative Comments:", negative_comments, f"({negative_percentage:.2f}%)")
print("Neutral Comments:", neutral_comments, f"({neutral_percentage:.2f}%)")
```

@Slip-26

Q.1)CreateemployeetableasfollowsEMP(eno,ename,designation,salary).WriteAjax programto Selecttheemployeesnameandprinttheselectedemployee’sdetails.

Ans:

<u>Htmlfile</u>

```
<!DOCTYPE html>
<html>
<head>
```

Edit with WPS Office

```
  <title>Employee Details</title>

  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

  <script>
  $(document).ready(function(){

    $("#employee-list").change(function(){

      var selectedEmployee = $(this).val();

      if (selectedEmployee == "") return;

      $.ajax({
        url: "empdetails.php",
        type: "POST",
        data: { employeeName: selectedEmployee },
        dataType: "json",

        success: function(response){

          var detailsHtml =
            "Employee Name: " + response.ename + "<br>" +
            "Designation: " + response.designation + "<br>" +
            "Salary: " + response.salary;

          $("#employee-details").html(detailsHtml);
        },

        error: function(){
          alert("Error fetching data");
        }
      });

    });

  });
  </script>

</head>

<body>

<h2>Select Employee</h2>

<select id="employee-list">
  <option value="">--Select Employee--</option>
  <option>John Doe</option>
```

Edit with WPS Office

```
    <option>Jane Smith</option>
    <option>Bob Johnson</option>
</select>

<br><br>

<div id="employee-details"></div>

</body>
</html>
```

Phpfileasempdetails.php

```
<?php

// Database connection (MySQL example)
$conn = mysqli_connect("localhost", "root", "", "test");

// Check connection
if (!$conn) {
    die("Connection failed");
}

// Get employee name
$employeeName = $_POST['employeeName'];

// Query
$sql = "SELECT * FROM emp WHERE ename='$employeeName'";
$result = mysqli_query($conn, $sql);

if (mysqli_num_rows($result) > 0) {

    $row = mysqli_fetch_assoc($result);

    $response = array(
        "ename" => $row['ename'],
        "designation" => $row['designation'],
        "salary" => $row['salary']
    );

    echo json_encode($response);

} else {
    echo json_encode(array("ename"=>"Not Found","designation"=>"","salary"=>""));
}

mysqli_close($conn);

?>
```

Edit with WPS Office

Database Table (SQL)

```
CREATE TABLE emp (
   eno INT PRIMARY KEY,
   ename VARCHAR(50),
   designation VARCHAR(50),
   salary INT
);

INSERT INTO emp VALUES
(1, 'John Doe', 'Manager', 50000),
(2, 'Jane Smith', 'Developer', 40000),
(3, 'Bob Johnson', 'Analyst', 35000);
```

Q.2)Considertextparagraph.“”” Helloall,WelcometoPythonProgrammingAcademy.Python ProgrammingAcademyisaniceplatformtolearnnewprogrammingskills.Itisdifficulttoget enrolled InthisAcademy.”””Preprocessthetexttoremoveanyspecialcharactersanddigits.Generate the Summaryusingextractivesummarizationprocess.Q.

Ans:

```
import re
import nltk
from nltk.tokenize import sent_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

nltk.download('punkt')

# Text to summarize
text = """Hello all, Welcome to Python Programming Academy.
Python Programming Academy is a nice platform to learn new programming skills.
It is difficult to get enrolled in this Academy."""

# Preprocessing: remove special characters and digits
clean_text = re.sub(r'[^a-zA-Z\s]', '', text)

# Sentence tokenization
sentences = sent_tokenize(clean_text)

# TF-IDF vectorization
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(sentences)

# Cosine similarity matrix
similarity_matrix = cosine_similarity(tfidf_matrix)

# Select sentence with highest similarity score
```

Edit with WPS Office

```
scores = similarity_matrix.sum(axis=1)

# Number of sentences to select in summary
N = 2

# Get top N sentences
top_sentence_indices = scores.argsort()[-N:][::-1]

# Generate summary
summary = " ".join([sentences[i] for i in top_sentence_indices])

print("Summary:\n")
print(summary)
```

@Slip-27

Q.1)CreatewebApplicationthatcontainsVotersdetailsandcheckpropervalidationfor(name, Age,andnationality),asNameshouldbeinuppercaselettersonly,Ageshouldnotbelessthan 18yrsandNationalityshouldbeIndian.(useHTML-AJAX-PHP).

Ans:

Htmlfile

```
<!DOCTYPE html>
<html>
<head>
   <title>Voter Details</title>

   <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>

   <script>
   $(document).ready(function(){

      $("#voterForm").submit(function(event){

         event.preventDefault();

         var name = $("#name").val();
         var age = $("#age").val();
         var nationality = $("#nationality").val();

         $.ajax({
            url: "voter.php",
            method: "POST",
            data: {
               name: name,
```

Edit with WPS Office

```
                age: age,
                nationality: nationality
            },

            success: function(response){
                $("#response").html(response);
            }
        });

    });

  });
  </script>

</head>

<body>

<h2>Voter Details</h2>

<form id="voterForm">

  <label>Name:</label>
  <input type="text" id="name" required>
  <br><br>

  <label>Age:</label>
  <input type="number" id="age" required>
  <br><br>

  <label>Nationality:</label>
  <input type="text" id="nationality" required>
  <br><br>

  <input type="submit" value="Submit">

</form>

<br>

<div id="response"></div>

</body>
</html>
```

Voter.phpfile

```
<?php
```

Edit with WPS Office

```
$name = $_POST['name'];
$age = $_POST['age'];
$nationality = $_POST['nationality'];

// Validate Name (uppercase only)
if (!preg_match('/^[A-Z ]+$/', $name)) {
   echo "Name should be in uppercase letters only.";
}

// Validate Age
else if ($age < 18) {
   echo "Age should not be less than 18 years.";
}

// Validate Nationality
else if (strcasecmp($nationality, "Indian") != 0) {
   echo "Nationality should be Indian.";
}

// Success
else {
   echo "Validation successful.<br>";
   echo "Name: $name <br>";
   echo "Age: $age <br>";
   echo "Nationality: $nationality";
}

?>
```

Q.2)Createyourowntransactionsdatasetandapplytheaboveprocessonyourdataset

Ans:

```
import random
import csv
import pandas as pd

# Generate random transaction data
transactions = []

for i in range(1, 101):
   transaction_id = i
   transaction_date = f"2022-05-{random.randint(1,31):02d}"
   customer_id = random.randint(1, 10)
   item_id = random.choice(["A", "B", "C"])
   item_price = round(random.uniform(10.0, 100.0), 2)
   quantity = random.randint(1, 10)
```

Edit with WPS Office

```
    transactions.append([
        transaction_id,
        transaction_date,
        customer_id,
        item_id,
        item_price,
        quantity
    ])

# Save to CSV file
with open('transactions.csv', 'w', newline='') as csvfile:
    writer = csv.writer(csvfile)
    writer.writerow([
        "TransactionID",
        "TransactionDate",
        "CustomerID",
        "ItemID",
        "ItemPrice",
        "Quantity"
    ])

    for transaction in transactions:
        writer.writerow(transaction)

# Read CSV into DataFrame
df = pd.read_csv('transactions.csv')

# Ensure numeric conversion
df['ItemPrice'] = pd.to_numeric(df['ItemPrice'])

# Calculate sales
df['Sales'] = df['ItemPrice'] * df['Quantity']

# Group by CustomerID and calculate total sales
total_sales = df.groupby('CustomerID')['Sales'].sum().reset_index()

# Print result
print(total_sales)
```

@Slip-28

Q.1)WriteaPHPscriptusingAJAXconcept,tocheckusernameandpasswordarevalidor Invalid(use Databasetostoreusernameandpassword).

Ans:

Edit with WPS Office

Htmlfile

```
<!DOCTYPE html>
<html>
<head>
  <title>Login</title>

  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>

  <script>
  $(document).ready(function(){

    $("#login-form").submit(function(event){

      event.preventDefault();

      var username = $("#username").val();
      var password = $("#password").val();

      $.ajax({
        url: "check_login.php",
        type: "POST",
        data: {
          username: username,
          password: password
        },

        success: function(response){

          if (response.trim() == "valid") {
            window.location.href = "dashboard.php";
          } else {
            alert("Invalid username or password");
          }
        }
      });

    });

  });
  </script>

</head>

<body>

<h2>Login</h2>
```

Edit with WPS Office

```
<form id="login-form">

   <label>Username:</label>
   <input type="text" id="username" required>
   <br><br>

   <label>Password:</label>
   <input type="password" id="password" required>
   <br><br>

   <input type="submit" value="Login">

</form>

</body>
</html>
```

Phpfileascheck_login.php

```
<?php

// Database connection
$conn = mysqli_connect("localhost", "root", "", "test");

// Check connection
if (!$conn) {
   die("Connection failed");
}

// Get data
$username = $_POST['username'];
$password = $_POST['password'];

// Query
$sql = "SELECT * FROM users WHERE username='$username' AND password='$password'";
$result = mysqli_query($conn, $sql);

// Check result
if (mysqli_num_rows($result) > 0) {
   echo "valid";
} else {
   echo "invalid";
}

mysqli_close($conn);

?>
```

Database Table (SQL)

Edit with WPS Office

```
CREATE TABLE users (
   id INT AUTO_INCREMENT PRIMARY KEY,
   username VARCHAR(50),
   password VARCHAR(50)
);

INSERT INTO users (username, password) VALUES
('admin', '1234'),
('user1', 'abcd');
```

Q.2)BuildasimplelinearregressionmodelforCarDataset.

Ans:

```
from sklearn.linear_model import LinearRegression

# Dataset (Independent variable: Mileage, Dependent variable: Price)
mileage = [[10], [20], [30], [40], [50], [60], [70], [80]]
price = [24, 19, 17, 13, 10, 7, 5, 2]

# Create and train model
reg = LinearRegression()
reg.fit(mileage, price)

# Print model parameters
print("Intercept:", reg.intercept_)
print("Coefficient:", reg.coef_[0])

# Predict for new mileage values
new_mileage = [[25], [45], [65]]
predicted_price = reg.predict(new_mileage)

print("Predicted Prices:", predicted_price)
```

@Slip-29

Q.1)WriteaPHPscriptforthefollowing:Designaformtoacceptanumberfromtheuser.
Performtheoperationsandshowtheresults.
1)FibonacciSeries.
2)Tofindsumofthedigitsofthatnumber.
(Usetheconceptofselfprocessingpage.)

Ans:

```
<!DOCTYPE html>
<html>
```

Edit with WPS Office

```
<head>
   <title>Number Operations</title>
</head>

<body>

<h2>Number Operations</h2>

<?php

$num = "";
$op = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {

   $num = test_input($_POST["num"]);
   $op  = test_input($_POST["op"]);

   if ($op == "fib") {
      $result = fibonacci($num);
      echo "<p>Fibonacci series up to $num terms: $result</p>";
   }
   else if ($op == "sum") {
      $result = sumOfDigits($num);
      echo "<p>Sum of digits of $num is: $result</p>";
   }
   else {
      echo "<p>Invalid operation selected</p>";
   }
}

// Function to sanitize input
function test_input($data) {
   $data = trim($data);
   $data = stripslashes($data);
   $data = htmlspecialchars($data);
   return $data;
}

// Fibonacci function
function fibonacci($num) {
   $first = 0;
   $second = 1;
   $result = "";

   for ($i = 0; $i < $num; $i++) {
      $result .= $first . " ";
      $third = $first + $second;
```

Edit with WPS Office

```
        $first = $second;
        $second = $third;
    }

    return $result;
}

// Sum of digits function
function sumOfDigits($num) {
    $sum = 0;

    while ($num > 0) {
        $digit = $num % 10;
        $sum += $digit;
        $num = (int)($num / 10);
    }

    return $sum;
}

?>

<!-- Self-processing form -->
<form method="post" action="<?php echo htmlspecialchars($_SERVER['PHP_SELF']); ?>">

    <label>Enter a number:</label>
    <input type="number" name="num" required>

    <br><br>

    <label>Select Operation:</label>
    <select name="op" required>
        <option value="">--Select--</option>
        <option value="fib">Fibonacci Series</option>
        <option value="sum">Sum of Digits</option>
    </select>

    <br><br>

    <input type="submit" value="Submit">

</form>

</body>
</html>
```

Q.2)BuildalogisticregressionmodelforStudentScoreDataset.

Edit with WPS Office

Ans:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load dataset
data = pd.read_csv('student_scores.csv')

# Split into input and output variables
X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)

# Create Logistic Regression model
classifier = LogisticRegression()
classifier.fit(X_train, y_train)

# Make predictions
y_pred = classifier.predict(X_test)

# Evaluate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

@Slip-30

Q.1)CreateaXMLfilewhichgivesdetailsofbooksavailablein“Bookstore”fromfollowing Categories.
1)Yoga
2)Story
3)Technical
Andelementsineachcategoryareinthefollowingformat
<Book>
<Book_Title>
----------</Book_Title>
<Book_Author>----------</Book_Author>
<Book_Price>
----------</Book_Price>
</Book>
Savethefileas“Bookcategory.xml”.

Edit with WPS Office

Ans:

Bookcategory.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<Bookstore>

  <!-- Yoga Category -->
  <Yoga>
    <Book>
      <Book_Title>Light on Yoga</Book_Title>
      <Book_Author>B.K.S. Iyengar</Book_Author>
      <Book_Price>20.99</Book_Price>
    </Book>

    <Book>
      <Book_Title>The Yoga Bible</Book_Title>
      <Book_Author>Christina Brown</Book_Author>
      <Book_Price>15.50</Book_Price>
    </Book>
  </Yoga>

  <!-- Story Category -->
  <Story>
    <Book>
      <Book_Title>The Alchemist</Book_Title>
      <Book_Author>Paulo Coelho</Book_Author>
      <Book_Price>12.99</Book_Price>
    </Book>

    <Book>
      <Book_Title>The Da Vinci Code</Book_Title>
      <Book_Author>Dan Brown</Book_Author>
      <Book_Price>14.75</Book_Price>
    </Book>
  </Story>

  <!-- Technical Category -->
  <Technical>
    <Book>
      <Book_Title>Python for Data Science Handbook</Book_Title>
      <Book_Author>Jake VanderPlas</Book_Author>
      <Book_Price>28.99</Book_Price>
    </Book>

    <Book>
      <Book_Title>Cracking the Coding Interview</Book_Title>
```

Edit with WPS Office

```
        <Book_Author>Gayle Laakmann McDowell</Book_Author>
        <Book_Price>23.50</Book_Price>
      </Book>
    </Technical>

</Bookstore>
```

Q.2)Createthedataset.transactions=[[‘eggs’,‘milk’,’bread’],[‘eggs’,‘apple’],[‘milk’,‘bread’], [‘apple’, ‘milk’],[‘milk’,‘apple’,‘bread’]].
Convertthecategoricalvaluesintonumericformat.Applytheapriorialgorithmontheabove datasetto Generatethefrequentitemsetsandassociationrules.

Ans:

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# Dataset
transactions = [
    ['eggs', 'milk', 'bread'],
    ['eggs', 'apple'],
    ['milk', 'bread'],
    ['apple', 'milk'],
    ['milk', 'apple', 'bread']
]

# Convert categorical data into numeric format (One-Hot Encoding)
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_array, columns=te.columns_)

# Apply Apriori algorithm
frequent_itemsets = apriori(df, min_support=0.4, use_colnames=True)

# Generate association rules
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)

# Print results
print("Frequent Itemsets:\n", frequent_itemsets)
print("\nAssociation Rules:\n", rules)
```

Edit with WPS Office